

SUPSI

MIG Music Generation Dashboard

Studente/i

Nicola Romano

Fabian Tomasovic

Relatore

Giorgia Adorni

Correlatore

Committente

ISIN

Corso di laurea

**Bachelor of Science in
Ingegneria informatica**

Modulo

M-P6202

Anno

2026

Data

4 maggio 2026

Indice

Abstract	1
1 Introduction	3
1.1 Context and Motivation	3
1.2 Starting Point	3
1.3 Project Goals	4
1.4 Document Structure	5
2 State of the art	7
2.1 Music Generation Models	7
2.2 Evaluation Metrics	8
3 System Overview	11
3.1 Technologies	11
3.1.1 Backend and API	11
3.1.2 Frontend	12
3.1.3 Pipeline libraries	12
3.2 Pipeline Stages	12
3.2.1 Infrastructure	13
4 System Architecture	15
4.1 Containerization	15
4.1.1 mig_backend	15
4.1.1.1 Dockerfile	15
4.1.1.2 docker-compose	17
4.1.2 mig_server	17
4.1.2.1 Dockerfile	17
4.1.2.2 docker-compose	18
4.1.3 mig_client	18
4.1.3.1 Dockerfile	18
4.1.3.2 docker-compose	18

4.1.4	ollama	18
4.1.4.1	Dockerfile	18
4.1.4.2	docker-compose	19
4.2	Repository structure	19
4.3	Host server architecture	20
4.4	Dependency Management	20
5	API and Run Management	21
5.1	Server API structure	21
5.1.1	Manage runs	21
5.1.2	Get information outside the run	22
5.1.3	Get specific run information	22
5.1.4	proxy to the backend	24
5.1.5	status	25
5.2	Backend API structure	25
5.2.1	Run pipeline stages	25
5.2.2	Monitor jobs	26
5.2.3	Music helpers	26
5.2.4	Status	27
5.3	Concurrency and run management	27
5.3.1	Run management	27
5.3.1.1	What is a run?	27
5.3.1.2	Initialising a run	27
5.3.1.3	Seeding a run	28
5.3.2	Concurrency in server requests	29
5.3.3	Concurrency in backend requests	29
5.3.4	Managed saga vs. direct step execution	30
5.3.4.1	Managed saga	30
5.3.4.2	Direct step proxies	30
6	Frontend	33
6.1	Structure Overview	33
6.1.1	Svelte and SvelteKit	33
6.1.2	Routing	33
6.1.3	Rendering	34
6.2	UI/UX	34
6.2.1	Design and Inspirations	34
6.3	Data Visualization	35
6.3.1	Design	35
6.3.2	Graph Generation	36

6.4	Backend Communication	36
6.4.1	API Fetching	36
6.5	Dashboard Screenshots	37
7	Integration and Deployment	39
7.1	Continuous Integration	39
7.1.1	Version Control System	39
7.1.1.1	Branching strategy	40
7.1.1.2	Protected environment variables	40
7.2	Continuous Deployment	40
7.2.1	Staging pipeline (dev branch)	41
7.2.1.1	unit-test	41
7.2.1.2	build:staging	42
7.2.1.3	deploy:staging	42
7.2.1.4	test:staging	42
7.2.2	Production pipeline (main branch)	42
7.2.3	On-Premises Server Requirements	43
7.3	Development workflow	43
8	Pipeline Update	47
8.1	Essentia to PANNs migration	47
8.1.1	Root cause	47
8.1.2	What Essentia provided	47
8.1.3	What PANNs provides	48
8.1.4	Migration steps and intermediate shim	48
8.1.5	Current schema	49
8.1.6	What PANNs cannot replicate	49
8.2	Handling edge cases	50
8.2.1	Missing instrumental stems	50
8.2.2	Generated audio file layout	50
8.2.3	Wikipedia API network failures	50
8.2.4	Ollama response failures in prompting services	51
9	Conclusions	53
9.1	Summary	53
9.1.1	Results	53
9.2	Limitations and future work	54
9.2.1	Dependency management in mig_backend	54
9.2.2	Incomplete model support	55
9.2.3	Monolithic backend container	55

9.2.4	Inversion of the job execution flow with Redis and Celery	55
9.2.5	API authentication and access control	56
9.2.6	Real-time log streaming	56
9.2.7	Frontend Error Prevention	56

Elenco delle figure

4.1 Architecture Diagram 16

5.1 Full pipeline execution sequence diagram 31

6.1 Colour Palette 35

6.2 Dashboard Home Page 35

6.3 Results Graph Mock-up 36

6.4 Fork Page 37

6.5 Enter Caption 37

7.1 Development workflow flowchart 45

Elenco delle tabelle

Abstract

English

The rapid proliferation of text-to-music generation models has created a need for systematic, reproducible frameworks to evaluate and compare their outputs under controlled conditions. This work presents the MIG Music Generation Dashboard (MMGD), a benchmarking platform that automates the full evaluation pipeline from reference audio to quantitative comparison of generated music.

The system implements a four-stage pipeline: vocal and instrumental stem separation, model-specific prompt generation through audio feature extraction and large language models, music synthesis across seven state-of-the-art generation models, and multi-dimensional metric computation combining CLAP score, tempo similarity, timbre similarity, and genre distribution alignment. All pipeline stages are isolated in dedicated Python virtual environments and executed on an NVIDIA Spark GPU server managed through a containerised Docker architecture.

A web dashboard built with Svelte provides researchers with an interactive interface to configure and launch runs, monitor execution status, inspect generated audio, and explore comparative visualisations of model performance. A CI/CD pipeline based on GitLab enforces strict separation between a staging environment for development and a production environment, ensuring reliable and reproducible deployments.

The platform enables structured experimentation with music generation models, reducing manual effort and providing a consistent basis for cross-model evaluation.

Italiano

La rapida proliferazione di modelli per la generazione musicale a partire da testo ha creato la necessità di framework sistematici e riproducibili per valutare e confrontare i loro output in condizioni controllate. Questo lavoro presenta il MIG Music Generation Dashboard (MMGD), una piattaforma di benchmarking che automatizza l'intero processo di valutazione, dall'audio di riferimento al confronto quantitativo della musica generata.

Il sistema implementa una pipeline in quattro fasi: separazione delle tracce strumentali e vocali, generazione di prompt specifici per modello tramite estrazione di caratteristiche audio e modelli linguistici di grandi dimensioni, sintesi musicale con sette modelli generativi allo stato dell'arte, e calcolo multidimensionale di metriche che combinano CLAP score, similarità di tempo, di timbro e di distribuzione di genere. Tutte le fasi della pipeline sono isolate in ambienti virtuali Python dedicati ed eseguite su un server GPU NVIDIA Spark gestito tramite un'architettura Docker containerizzata.

Una dashboard web sviluppata con Svelte offre ai ricercatori un'interfaccia interattiva per configurare ed avviare esecuzioni, monitorarne lo stato, ascoltare l'audio generato ed esplorare visualizzazioni comparative delle prestazioni dei modelli. Una pipeline CI/CD basata su GitLab garantisce una separazione netta tra ambiente di staging e ambiente di produzione, assicurando deployment affidabili e riproducibili.

La piattaforma consente una sperimentazione strutturata con modelli di generazione musicale, riducendo lo sforzo manuale e fornendo una base coerente per la valutazione comparativa tra modelli.

Capitolo 1

Introduction

1.1 Context and Motivation

The field of automatic music generation has undergone rapid transformation in recent years. Advances in deep learning, and in particular in diffusion models and large language models, have produced a new generation of systems capable of synthesising high-quality, stylistically coherent audio from natural language descriptions. Models such as MusicGen, AudioLDM2, ACE-Step, and Stable Audio Open have demonstrated remarkable capabilities, but the diversity of their architectures, training data, and prompt formats makes direct comparison difficult. There is no standard benchmark that evaluates these models under controlled conditions using the same input songs, the same prompts, and the same set of objective metrics.

This is what motivated the development of an internal evaluation pipeline capable of comparing multiple generation models side by side. This project was first and is still under development by the project relator Giorgia Adorni, under the name of MultiInteractGen (MIG). We entered the project in February 2026 with the goal of engineering and deploying a web app based on the work already done by the project relator.

1.2 Starting Point

When we joined the project, the foundation had already been laid by the project relator. The existing codebase consisted of a set of Python scripts organised into four pipeline stages: stem separation, prompt generation, music generation, and metric computation. These scripts were designed to be run sequentially and manually on her x86 workstation, with results written to a local `/output/` directory.

Each pipeline stage had its own `requirements.txt` file to manage the conflicting dependencies between the different machine learning libraries involved. Runs were triggered by executing the scripts directly from the terminal, with output paths and parameters configured by editing the scripts themselves or passing command line arguments. There was no web interface, no API, and no automated deployment.

While the pipeline logic was functionally sound, this setup presented several limitations that became the starting point for our work:

- **No reproducibility across environments.** The scripts ran on a specific workstation configuration and had never been tested on any other machine. Dependencies in some cases needed to be updated, and the pipeline had no containerisation.
- **No usable interface.** Interacting with the pipeline required direct access to the terminal and knowledge of the script parameters. Results were raw CSV files and audio files in a folder hierarchy, with no way to browse or compare them easily.
- **No experiment management.** Running the pipeline multiple times with different models or parameters would overwrite previous results unless the output directory was manually renamed beforehand.
- **Tied to a single machine.** The target deployment environment was an NVIDIA Spark server, an aarch64 (ARM64) machine, which the pipeline had never been tested on. As we discovered during integration, this introduced a critical incompatibility in the metrics stage that required a non-trivial migration effort.

1.3 Project Goals

Given this starting point, our work focused on two parallel tracks: evolving the existing pipeline into a robust, deployable system, and building the tooling and interfaces that would make it usable by researchers beyond the original development environment.

Concretely, the goals were:

- **Containerisation and portability.** Package the pipeline and all its dependencies into Docker [1] containers, ensuring that the system runs identically across compatible environments and can be deployed and updated reliably.
- **API and run management.** Design and implement a server layer that exposes the pipeline stages as HTTP endpoints, manages experiment runs with unique identifiers and structured metadata, and handles concurrent execution safely under the constraint of a single shared GPU.

- **Web dashboard.** Build an interactive frontend that allows researchers to configure and start runs, monitor their progress, listen to generated audio, and compare models through visual metric summaries.
- **CI/CD pipeline.** Establish a development workflow with automatic testing and deployment to both a staging and a production environment, enabling safe iteration without risk to the live system.
- **Platform migration and pipeline stabilisation.** Adapt and fix the pipeline to run correctly on the aarch64 deployment server, resolving dependency incompatibilities and edge cases discovered during integration.

1.4 Document Structure

The remainder of this document is organised as follows. Chapter 2 surveys the state of the art in music generation and evaluation metrics. Chapter 3 gives a high-level overview of the system and the technologies used. Chapter 4 describes the system architecture and containerisation strategy. Chapter 5 covers the API design and run management logic. Chapter 6 documents the web frontend. Chapter 7 details the CI/CD pipeline and deployment infrastructure. Chapter 8 describes the platform migration from x86 to aarch64 and the pipeline fixes applied during integration. Chapter 9 concludes with a summary of results, known limitations, and directions for future work.

Capitolo 2

State of the art

2.1 Music Generation Models

Text-to-music generation models take a natural language description as input and produce an audio waveform as output. The dominant architectures fall into two families: autoregressive language models operating on discrete audio tokens, and diffusion models operating in either the waveform or a latent space.

MusicGen [2] is an autoregressive transformer developed by Meta AI that generates music from text prompts and, optionally, a melodic conditioning signal. It operates on EnCodec [3] audio tokens and is available in several sizes. Its single-stage architecture without cascading models makes it straightforward to integrate and was one of the first open models to achieve competitive quality at controllable duration.

AudioLDM 2 [4] is a latent diffusion model trained on a unified representation that covers both speech and general audio, including music. It conditions generation on text via a language model that maps the prompt into a shared AudioMAE latent space, enabling cross-modal transfer. Its unified design distinguishes it from earlier AudioLDM [5] and makes music generation one of several audio tasks it supports.

MusicLDM [6] adapts the latent diffusion approach specifically for music by training on a music-only dataset and replacing the CLAP text encoder with one fine-tuned on music captions. It also introduces beat-tracking based mixup strategies during training to improve stylistic coherence.

Beyond the three models evaluated in this project, the platform also provides integration with **Stable Audio Open** [7], **InspireMusic** [8], **YUE** [9], and **ACE-Step** [10], covering a broader spectrum of architectures including flow matching and step-based generation. These were integrated into the system and are already supported by the codebase, but not included in the evaluation runs documented in this report, as their dependency requirements require further validation on the deployment hardware.

2.2 Evaluation Metrics

Evaluating text-to-music generation requires analysing and taking into account multiple and diverse aspects of the generated output. A generated track may be acoustically convincing but stylistically distant from the reference; it may match the prompt semantically but differ in tempo or instrumentation. MMGD combines four complementary metrics into a single `combined_kpi` to capture different aspects of generation quality.

CLAP score. The Contrastive Language-Audio Pretraining model [11] learns a joint embedding space for audio and text. The CLAP score measures the cosine similarity between the embedding of a generated audio clip and the embedding of the text prompt used to generate it, providing a direct measure of semantic alignment. MMGD uses the `630k-audioset-fusion-best.pt` checkpoint from LAION-CLAP [12], which supports variable-length inputs via a fusion mechanism and was trained on a large and diverse audio dataset.

Tempo similarity. Beats per minute (BPM) is one of the most fundamental structural properties of a musical track. Tempo is extracted from both the reference instrumental and the generated audio using **librosa** [13] and compared as a normalised similarity score. A generated track that matches the reference in tempo is more likely to be perceived as stylistically consistent.

Timbre similarity. Timbre captures the tonal colour of a recording - the combined effect of instrument choice, production style, and spectral distribution. MMGD extracts Mel-frequency cepstral coefficients (MFCCs) and spectral features from both tracks and computes their cosine similarity as a proxy for timbral closeness [14].

Genre similarity (PANNs cosine similarity). Genre classification is used to verify that the generated audio belongs to the same stylistic space as the reference. MMGD uses **PANNs** (Pre-trained Audio Neural Networks) [15], a model trained on AudioSet with 527 output classes, to extract a probability distribution over sound categories for both tracks. The cosine similarity between these two distributions provides a continuous measure of genre agreement. Additionally, Jaccard similarity over the top predicted music-relevant tags provides a discrete counterpart.

The four metrics are averaged into the `combined_kpi`:

$$\text{combined_kpi} = \frac{\text{clap_score} + \text{tempo_similarity} + \text{timbre_similarity} + \text{panns_cosine_similarity}}{4} \quad (2.1)$$

Low-quality generations are filtered out before aggregation using the CLAP score on the original reference music as a quality threshold, removing tracks where the prompt itself does not adequately describe the source material.

Evaluation corpus

The reference tracks used in the evaluation were selected by the project relator and span a wide range of genres, languages, and eras, from Miles Davis's *So What* (1959) to Anyma's *Eternity* (2023). The corpus includes jazz, classical, rock, electronic, reggae, pop, and K-pop, in English, Italian, Japanese, Korean, and Spanish. This diversity is intentional: a robust benchmarking pipeline should produce meaningful comparisons regardless of the musical style of the reference track, and the breadth of the corpus helps surface model-specific weaknesses in stylistic coverage.

Capitolo 3

System Overview

MIG Music Generation Dashboard is a benchmarking platform designed to evaluate and compare the performance of different text to music generation models against audio tracks. The app takes in input one or more audio tracks, and then automatically extracts audio stems, audio descriptors, produces synthetic audio with the chosen music generation models, calculates and serves metrics to compare the output between the models and against the original audio.

The results are presented in a web dashboard that allows researches to inspect runs, compare models and access the evaluation data interactively.

The application is deployed via a CI/CD pipeline as a set of docker containers on an NVIDIA Spark server. The application can be deployed in two separate environment for staging and production. The design prioritises reproducibility, pipeline isolation and safe concurrent execution under constraint of a single shared GPU.

3.1 Technologies

3.1.1 Backend and API

- **Python**: for the server side logic.
- **FastAPI** [16]: for both the public-facing API (`mig_server`) and the internal pipeline executor (`mig_backend`), which provides asynchronous request handling, automatic schema generation via Pydantic, and interactive Swagger documentation at `/docs`.
- **uv** [17]: for dependency management in `mig_server`.
- **pip**: for dependency management in `mig_backend`.

3.1.2 Frontend

- **SvelteKit** [18]: for templating/rendering the dashboard and for application routing.
- **JavaScript**: for client-side logic.
- **Vite**: for proxy management and website scaffolding.
- **npm**: for dependency management and local testing.

3.1.3 Pipeline libraries

Each pipeline stage uses a different set of specific libraries:

- **Stage 1, Stem separation**: **htdemucs** (Meta Demucs) [19] for local GPU-accelerated source separation, with an optional cloud fallback via the **lalal.ai** API. PyTorch with CUDA support is required.
- **Stage 2, Prompt generation**: **Whisper** [20] for transcription, **AcoustID** and **librosa** for audio fingerprinting and feature extraction, and the **MusicBrainz**, **Wikipedia**, and **Genius** APIs for metadata enrichment. Prompt synthesis is handled by **OpenAI GPT** [21]; instrument and epoch classification use a local **Mistral** [22] model served by **Ollama** [23].
- **Stage 3, Music generation**: a wrapper architecture that supports three open-source generation models, **MusicGen**, **MusicLDM**, **AudioLDM2**, and can be updated to support **Stable Audio Open**, **InspireMusic**, **YUE**, and **ACE-Step**: loaded from a shared model weights directory to avoid duplication across environments.
- **Stage 4, Metric computation**: **librosa** for audio descriptors, **PANNs** (Pre-trained Audio Neural Networks) for genre classification, and **CLAP** for cross-modal audio–text similarity scoring. Metric aggregation and chart generation use **pandas** [24] and **Altair** [25].

3.2 Pipeline Stages

The core of MMGD is a four-stage benchmarking pipeline. Every execution is grouped into a **run**: a self-contained directory under `output/runs/{run_id}/` that holds all inputs, intermediate outputs, logs, and metadata for that execution. Stages are executed sequentially; each stage reads from and writes to the shared run directory, so the outputs of stage N are always available to stage $N+1$ before it begins.

1. **Stem separation.** The reference audio tracks are separated into individual stems (vocals, bass, drums, and other instruments) using **htdemucs**. The non-vocal stems

are then merged into a single `_instrumental.wav` file per track, which serves as the clean reference signal for all subsequent stages.

2. **Prompt generation.** For each reference track, the pipeline extracts a rich set of audio and contextual descriptors, BPM, key, timbre, instrumentation, genre, era, lyrics, and more, by combining local audio analysis with external metadata APIs. These descriptors are stored in `descriptors.csv` and then used by an LLM to synthesise model-specific text prompts optimised for each of the seven supported generation models.
3. **Music generation.** Using the prompts produced in stage 2, each selected generation model produces one or more audio files per reference track. The stage runs once per model: to benchmark multiple models, one job is submitted per model and they are serialised through the GPU executor. Generated audio is written to `music/open_generation/{model_name}/`.
4. **Metric computation.** The generated audio is evaluated against the original reference tracks using four complementary metrics: CLAP score (prompt-to-audio semantic alignment), tempo similarity, timbre similarity, and PANNs cosine similarity (genre distribution alignment). These are combined into a single `combined_kpi` per generation. Low-quality generations are removed through successive filtering passes, and results across all models are aggregated for cross-model comparison.

A run can be started from any stage by **seeding** it from a prior completed run, allowing researchers to reuse stem separation and prompt generation results when comparing new generation models without repeating the time-consuming earlier stages. The pipeline is also resumable within a stage: if a run is interrupted, already-produced outputs are detected and skipped on restart

3.2.1 Infrastructure

- **Docker** [1]: for the containerization.
- **Docker Compose** [26]: for the containers orchestration.
- **GitLab CI** [27] as VCS and for the continuous integration and deployment.

Capitolo 4

System Architecture

MMGD is a multi container application. There are four separate containers that collaborate over a private network.

The container are deployed and run on an NVIDIA-SPARK. For more details see chapter 7 Integration and Deployment.

4.1 Containerization

The four containers of which the application is composed are:

- **mig_backend**: Pipeline executor and "jobs" manager. Hosts the FastAPI server (api.py) on port 8000. Execute pipeline steps as subprocesses. Each pipeline is executed in a dedicated virtual environment.
- **mig_server**: Public FastAPI application, exposing high level request acting as ingress point. Proxies pipelines jobs to the backend. Manages runs execution.
- **mig_client**: a Svelte multi page web app served via Vite.
- **ollama**: On-Premises LLM inference server hosting Mistral and Ollama.

These containers rely on some specific files and infrastructure. For more details see chapter 7 Integration and Deployment.

4.1.1 mig_backend

4.1.1.1 Dockerfile

This container is built starting from the **Dockerfile.backend**.

Base image: nvidia/cuda:13.0.0-devel-ubuntu22.04.

This base image was chosen as it comes with cuda pre-installed.

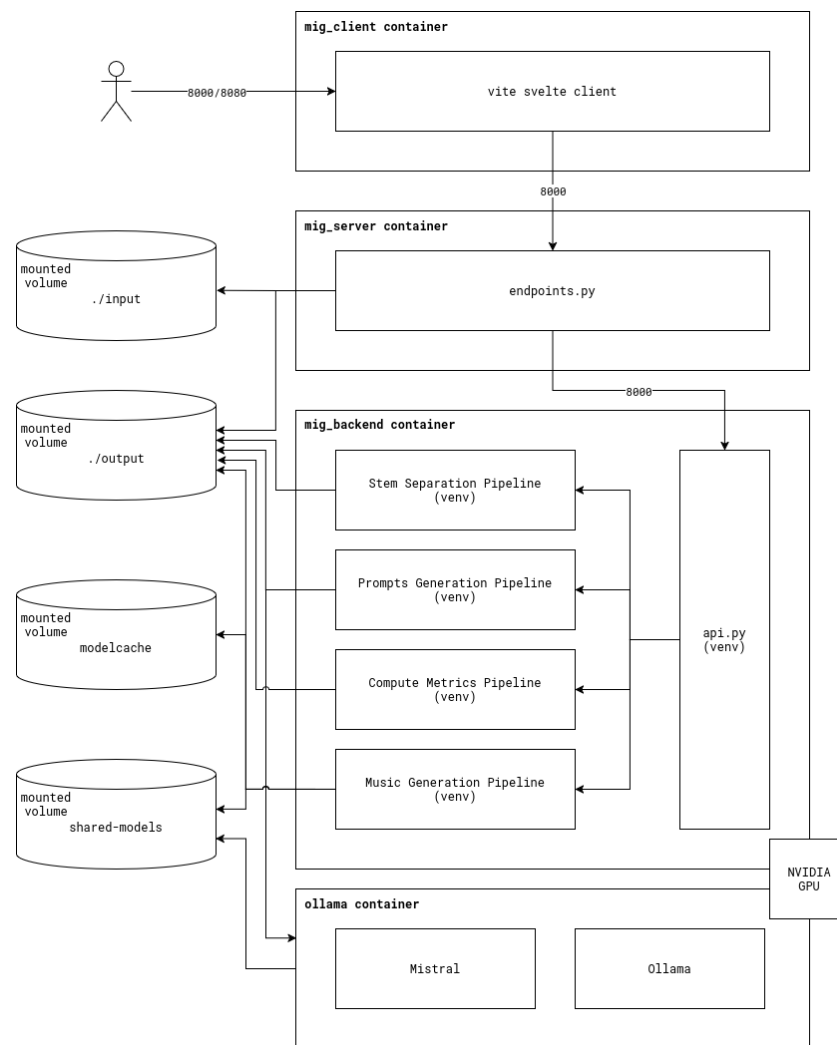


Figura 4.1: Architecture Diagram

The aforementioned Dockerfile installs directly on the container python3.10 (set as default) and other libraries necessary for the pipeline execution, for example ffmpeg. (more information at chapter 4.4 Dependency management)

The mig_backend container hosts 5 python virtual environments:

- /opt/venv_stems
- /opt/venv_prompts
- /opt/venv_music
- /opt/venv_metrics
- /opt/venv_api

Each of these virtual environments is used to execute one pipeline (the only exception is `/opt/venv_api`, used to run the FastAPI application. The requirements used to create the pipeline `venvs` can be found under `envs/<n-pipeline-name>/requirements.txt`.

The separation in five different `venvs` was necessary due to conflicts in the dependencies of the different pipelines. A Cleaner approach for the future is discussed in chapter 9.2 Limitation and future work.

4.1.1.2 docker-compose

As defined in the `docker-compose.yml` and `docker-compose.<staging/production>.yaml` this container mount three volumes:

- `./output:/app/output`: Here the pipeline will find the runs created by the server, with all the data necessary to their execution.
- `.../Desktop/shared-models/open:/app/backend/src/models`: The models weights and parameters, shared between all containers and environments (production and staging, more information at chapter 7.2 Continuous Deployment)
- `model_cache:/root/.cache`: models cache, set as external, so that isn't lost between container builds, as it would be redundant to populate it from scratch every time.

The container is configured to have direct access to the server nvidia gpu. This is necessary for the correct execution of the pipelines.

The container is configured to restart unless explicitly stopped.

The differences between how the container is built in staging and production are discussed at chapter 7 section 2: Continuous Deployment.

An environment variable `BACKEND_PORT` is defined and assigned value:8000. This will be the port exposed by the FastAPI server (`api.py`).

4.1.2 mig_server

4.1.2.1 Dockerfile

This container is built starting from the `Dockerfile.server`.

Base image: `alpine:3.23`.

This base image was chosen as its particularly light. As we don't have other specific necessity we privileged a light and easy to maintain image.

This container uses `uv` to install the necessary dependencies for the FastAPI server (`endpoints.py`).

The server is then launched by the `entrypoint.sh`

4.1.2.2 docker-compose

As defined in the **docker-compose.yml** and **docker-compose.<staging/production>.yaml** this container mount three volumes:

- **./input/app/input**: Here live the input songs, the selected input song will be copied in the **output/runs/<runid>/input** directory.
- **./output/app/output**: Here the server initializes and manages the runs, creating and moving all the necessary files for the pipelines to execute.
- **.../Desktop/shared-models/open:/app/backend/src/models**: The models weights and parameters, shared between all containers and environments (production and staging, more information at chapter 7.2 Continuous Deployment)

The container is configured to restart unless explicitly stopped.

The differences between how the container is built in staging and production are discussed at chapter 7.2 Continuous Deployment.

An environment variable **BACKEND_URL** is defined and assigned value:8000. This is the port at which the server will try to contact the backend API, therefore it must match the **BACKEND_PORT** variable defined in the backend service, (see 4.1.1.2)

4.1.3 mig_client

4.1.3.1 Dockerfile

This container is built starting from the **Dockerfile.client**.

Base image: **alpine:3.23**.

This base image was chosen as its particularly light. As we don't have other specific necessity we privileged a light and easy to maintain image.

This container uses **npm** to handle dependency management, and is run using **npx vite**, which is a default alias for **npm run dev**.

4.1.3.2 docker-compose

As defined in the **docker-compose.yml** and **docker-compose.<staging/production>.yaml** this container mount no volumes. It exposes the port **8080:8080** for staging and **8080:80** for production.

The container is configured to restart unless explicitly stopped.

4.1.4 ollama

4.1.4.1 Dockerfile

This container is defined purely from the **docker-compose**, as the image is pulled used as-is.

4.1.4.2 docker-compose

As defined in the **docker-compose.staging.yml** and **docker-compose.prod.yml** this container one volume:

- `.../Desktop/shared-models/open:/app/backend/src/models`: The models weights and parameters, shared between all containers and environments (production and staging, more information at chapter 7.2 Continuous Deployment)

The container is configured to have direct access to the server nvidia gpu. This is necessary for the correct execution of the pipelines.

The container is configured to restart unless explicitly stopped.

The default port where ollama serves its endpoints (11434) is not overwritten, so its services are exposed there.

4.2 Repository structure

```

mig-musicgen-dashboard/
|-- backend/src/
|   |-- api.py                # Internal FastAPI (port 8001)
|   |-- pipelines/           # Four pipeline entry-point scripts
|   |-- models/open/        # Per-model generation wrappers
|   |-- metrics/, prompting/, stemmer/, utils/
|-- server/src/
|   '-- endpoints.py         # Public FastAPI (port 8000)
|-- client/
|   |-- dist/
|   |-- static/              # Static Files (Images, Audios...)
|   |-- src/
|   |-- vite.config.js, package.json, jsconfig.json, ...
|-- server/src/
|   '-- endpoints.py         # Public FastAPI (port 8000)
|-- containers/              # Dockerfiles and entrypoints
|-- envs/                    # Per-stage requirements.txt
|-- input/                   # Reference audio (git-ignored)
|-- output/                  # Run artifacts (git-ignored)
|-- docker-compose.yml       # Shared base compose file
|-- docker-compose_staging.yml
|-- docker-compose_prod.yml
'-- sync-staging.sh          # sync automation script

```

4.3 Host server architecture

The production host (an NVIDIA Spark GPU server) uses a strict three-directory layout to prevent Git lock collisions and Docker permission errors during CI/CD deployments:

- `shared-models/`: single source of truth for all heavy AI model weights. Both staging and production containers access this directory, avoiding duplicated gigabytes of weights.
- `staging-env/`: the active Git repository used by the GitLab Runner. Live source code is bind-mounted into the staging containers.
- `prod-deployment/`: a lean, non-Git directory containing only the production `.env`, `input/`, `output/`. The GitLab runner will build here the production containers.

4.4 Dependency Management

Different containers are using different solutions to manage dependencies.

- **mig_backend**: Some libraries are directly installed in the container: `python3.10`, `ffmpeg`, `wget`, ...
These dependency version are not pinned, and being that are installed in the container are shared between all the virtual environments.
Python dependencies are handled with **pip**.
The requirements for the single virtual environment are specified in the `envs/<n-pipeline-name>/requirements.txt` file.
These requirement files pin the version of the dependency to guarantee reproducibility across different builds. The `index url` <https://download.pytorch.org/whl/cu130> is provided to install specific torch, TorchVision and TorchAudio libraries for CUDA.
Problems in handling dependencies with pip are discussed at chapter 9.2 limitations and future work.
- **mig_server**: All dependencies other than `python3`, `py3-pip`, `curl`, `build-base` and `uv` are handled via **uv** [17].
UV creates a `uv.lock` files where dependencies of first and higher degree are pinned, so that the environment is consistent between builds.
- **mig_client**: All dependencies are managed using `npm`, which creates a `package.json` file to synchronize dependencies, with their relative version constraints. Building and running the application will synchronize dependencies automatically.

Capitolo 5

API and Run Management

The system exposes two HTTP APIs, both implemented with FastAPI:

- **mig_server** public API. Is the only surface reachable outside the Docker network. An interactive swagger documentation is available at `/api/docs`.

The production and staging environments expose this api at two different ports:

- Staging base URL: `http://localhost:8080`
- Production base URL: `http://localhost:80`

- **mig_server** API. Is exposed only inside the Docker network.
Is reachable at the internal base URL: `http://backend:8000`. This port could be changed, the constraint to do so are specified at section 4.1.1.2 (mig-backend, docker-compose) and section 4.1.2.2 (mig-client, docker-compose), (look for the `BACKEND_URL` and `BACKEND_PORT` are handled).

5.1 Server API structure

5.1.1 Manage runs

These API request are used to manage the lifecycle of a run. A run is a execution one or more pipeline steps in sequence, given a specific set of input and parameters. Runs are handled by the `mig_backend` container. For more information see chapter 5.3 Concurrency and run management.

- POST `/pipeline/full` – Start Pipeline
- GET `/pipeline/full/schema`
Get Pipeline Full Schema
- POST `/runs/init` – Init Run
- DELETE `/runs/run_id:-` Delete Run

5.1.2 Get information outside the run

These get request can be used to retrieve information not strictly related to a run

- GET `/models`
Get Available Models
- GET `/songs`
Get Available Songs
- GET `/runs`
List Runs

5.1.3 Get specific run information

These request can be used to retrieve specific run information, which lives in the `/output/runs/<run_id>/` directory.

- GET `/runs/{run_id}`
Get Run information, serialized in the `run.json`.
- GET `/runs/{run_id}/seed-options`
Get Seed Options, which are the possible steps from which you can start a forked run.
- GET `/runs/{run_id}/log`
Get Run Log.
- GET `/runs/{run_id}/prompts`
Get Run Prompts.
- GET `/runs/{run_id}/songs/descriptions`
Get Available Songs Descriptions
- GET `/runs/{run_id}/charts/comparison`
Get Comparison Chart, generated with altair-vega
- GET `/runs/{run_id}/charts/generations`
Get Generations Chart, generated with altair-vega
- GET `/runs/{run_id}/rawdescriptors/default`
Get Run Default Descriptors
- GET `/runs/{run_id}/rawdescriptors/generated`
Get Run Descriptors On Generated Music
- GET `/runs/{run_id}/rawdescriptors/original`
Get Run Descriptors On Original

- GET /runs/{run_id}/rawdescriptors/all
Get Run All Descriptors
- GET /runs/{run_id}/rawmetrics/
aggregated/default
Get Raw Metrics Aggregated Default, as csv lines
- GET /runs/{run_id}/rawmetrics/
aggregated/filtered
Get Raw Metrics Aggregated Filtered, as csv lines
- GET /runs/{run_id}/rawmetrics/
aggregated/filterfiltered
Get Raw Metrics Aggregated Filterfiltered, as csv lines
- GET /runs/{run_id}/rawmetrics/
aggregated/alternative/filtered
Get Raw Metrics Aggregated Alternative Filtered, as csv lines
- GET /runs/{run_id}/rawmetrics/
aggregated/final
Get Raw Metrics Aggregated Final, as csv lines
- GET /runs/{run_id}/rawmetrics/
aggregated/alternative/filterfiltered
Get Raw Metrics Aggregated Alternative Filterfiltered,as csv lines
- GET /runs/{run_id}/rawmetrics/
aggregated/alternative/final
Get Raw Metrics Aggregated Alternative Final, as csv lines
- GET /runs/{run_id}/rawmetrics/
originalmusic/clap/{model}
Get Raw Metrics Originalmusic Clap For Model , as csv lines
- GET /runs/{run_id}/rawmetrics/
originalmusic/filterclap/{model}
Get Raw Metrics Originalmusic Filterclap For Model, as csv lines
- GET /runs/{run_id}/rawmetrics/
generatedmusic/clap/{model}
Get Raw Metrics Generatedmusic Clap For Model, as csv lines

- GET /runs/{run_id}/rawmetrics/
generatedmusic/descriptors/{model}
Get Raw Metrics Generatedmusic Descriptors For Model, as csv lines
- GET /runs/{run_id}/rawmetrics/
generatedmusic/merged/{model}
Get Raw Metrics Generatedmusic Merged For Model, as csv lines
- GET /runs/{run_id}/rawmetrics/
generatedmusic/filtercombined/{model}
Get Raw Metrics Generatedmusic Filtercombined For Model, as csv lines
- GET /runs/{run_id}/rawmetrics/
generatedmusic/filterfilteredcombined/{model}
Get Raw Metrics Generatedmusic Filterfilteredcombined For Model, as csv lines
- GET /runs/{run_id}/rawmetrics/
generatedmusic/alternativefiltercombined/{model}
Get Raw Metrics Generatedmusic Alternativefiltercombined For Model, as csv lines
- GET /runs/{run_id}/rawmetrics/
generatedmusic/alternativefilterfilteredcombined/{model}
Get Raw Metrics Generatedmusic Alternativefilterfilteredcombined For Model, as csv
lines

, as csv lines

5.1.4 proxy to the backend

These requests are proxies to the backend, as they regard jobs (backend concern) or are specific pipeline step execution without server intervention. The direct step request could break a run structure, so should be used carefully. They are not used by the UI, but can still be used via the swagger or other tools to make http request. The correct choice of parameters to keep the run consistent is up to the user.

- GET /jobs
Get Jobs List
- GET /jobs/{job_id}
Get Job
- POST /pipeline/stems
Stems, single step execution, no run

- GET /pipeline/stems/schema
Get Stems Schema
- POST /pipeline/prompts
Prompts, single step execution, no run
- GET /pipeline/prompts/schema
Get Prompts Schema
- POST /pipeline/music
Music, single step execution, no run
- GET /pipeline/music/schema
Get Music Schema
- POST /pipeline/metrics
Metrics, single step execution, no run
- GET /pipeline/metrics/schema
Get Metrics Schema

5.1.5 status

This request can be used to check the status of the server. Is used by the CI/CD pipelines as discussed in chapter 7. Integration and Deployment

- GET /health
Health check, just returns 200

5.2 Backend API structure

The backend API is a FastAPI application exposed exclusively inside the Docker network at `http://backend:8000`. It is never reachable from outside and is consumed solely by the **mig_server**. Its sole responsibilities are executing pipeline stages as subprocesses and tracking the resulting jobs in memory. It does not write to `run.json`: that is the server's concern.

5.2.1 Run pipeline stages

These endpoints accept a stage request and immediately submit the corresponding subprocess to a single-worker executor (enforcing GPU serialisation). They return a `job_id` that the server can poll via the job endpoints below.

- POST /pipeline/stems
Stems

- GET /pipeline/stems/schema
Get Stems Schema
- POST /pipeline/prompts
Prompts
- GET /pipeline/prompts/schema
Get Prompts Schema
- POST /pipeline/music
Music
- GET /pipeline/music/schema
Get Music Schema
- POST /pipeline/metrics
Metrics
- GET /pipeline/metrics/schema
Get Metrics Schema

Each POST endpoint accepts a `run_path` field (the absolute path to the run directory already prepared by the server) in addition to the stage-specific parameters described in chapter 5 – Pipelines.

5.2.2 Monitor jobs

Jobs are stored in memory for the lifetime of the backend process. The server polls these endpoints to determine when a stage has completed and whether it succeeded.

- GET /jobs
Get Jobs List
- GET /jobs/{job_id}
Get Job

A job object contains a `status` field (`pending` → `running` → `done` | `error`) and, on failure, a `detail` field with the subprocess exit code or exception message.

5.2.3 Music helpers

- POST /stems/ensure-instrumentals
Ensure Instrumentals

Re-creates any missing `_instrumental.wav` files by summing the existing bass, drums and other stems for each song in the run. Accepts a `run_path` query parameter. Called automatically by the server saga before stage 4 (metrics); can also be triggered manually to repair a run without re-executing the full stems stage.

5.2.4 Status

- GET `/health`
Health

Returns `{"status": "ok", "service": "backend"}`. Used by `wait_for_health.sh` to verify the backend container is ready before the server begins forwarding requests, as discussed in chapter 7. Integration and Deployment.

5.3 Concurrency and run management

5.3.1 Run management

5.3.1.1 What is a run?

A **run** is the unit of work in MMGD. Every pipeline execution is self-contained in its own directory under `/app/output/runs/{run_id}/`, which holds the selected input songs, all stage outputs, a `pipeline.log`, and a `run.json` metadata file tracking the overall status and the per-step status.

`run_id` values are auto-generated by the server at initialisation time, combining a UTC timestamp with a 6-character hex suffix:

YYYYMMDDTHHmmSS_xxxxxx e.g. 20250114T103201_a3f9c1

5.3.1.2 Initialising a run

Run initialisation is performed exclusively by the server (never the backend). When `POST /pipeline/full` or `POST /runs/init` is called, the server:

1. Generates a unique `run_id` and creates the run directory.
2. Copies the user-selected songs from the global library (`/app/input/music/`) into `/app/output/runs/{run_id}/input/music/`.
3. Writes `run.json` with all step statuses set to pending.

If initialisation fails at any point, the partially created run directory is deleted to avoid leaving inconsistent state on disk.

The `run.json` file is the single source of truth for run state. Its schema is:

```
{
  "run_id":      "20250114T103201_a3f9c1",
  "created_at":  "2025-01-14T10:32:01.000000+00:00",
  "status":      "done",
  "start_step":  "stems",
  "seeded_from": null,
  "model_names": ["music_gen", "ace_step"],
  "song_names":  ["my_song.wav"],
  "step_status": {
    "stems":      "done",
    "prompts":    "done",
    "music":       "done",
    "metrics":     "done"
  },
  "error_detail": null
}
```

The overall status field progresses as: pending → running → done | error. Each entry in `step_status` follows the same states, with the addition of seeded for steps that were copied from a prior run (see section 5.3.3).

The backend never writes to `run.json`. Only the server updates it, after each stage either completes or fails.

5.3.1.3 Seeding a run

Seeding allows a new run to reuse the outputs of a prior run for stages that have already been completed, starting the pipeline from a later step. This is particularly useful when comparing different generation models without repeating the time-consuming stem separation and prompt generation phases.

To seed a run, `POST /pipeline/full` is called with a `seeded_from` field set to the `run_id` of the source run and a `start_step` set to the stage where execution should begin. The server then:

1. Validates that all stages preceding `start_step` are in done or seeded status in the source run.
2. Copies the relevant output directories from the source run into the new run directory (e.g. `stems/`, `prompts/`, `descriptors/` if `start_step` is `music`).
3. Copies the input songs from the source run (the `song_names` field in the request is ignored when seeding).
4. Sets the `step_status` of all copied stages to `seeded` in the new `run.json`.

The endpoint `GET /runs/{run_id}/seed-options` can be used to check which steps of a given run are available as seed points before submitting a new request.

5.3.2 Concurrency in server requests

The server is an **async** FastAPI application. Request reception and handling is natively concurrent. When `POST /pipeline/full` is received, the server initialises the run synchronously (generating the `run_id`, creating the directory, writing `run.json`), then spawns the saga orchestrator as a background `asyncio` task via `asyncio.create_task()` and immediately returns the `job_id` and `run_id` to the caller.

This means the server can accept and begin processing a second `POST /pipeline/full` request while a saga is already running in the background. Each saga runs independently in its own `asyncio` task and manages the lifecycle of a separate run. The two runs do not interfere with each other at the server level.

However, because both sagas will eventually submit GPU-heavy jobs to the backend, and the backend serialises all jobs through a single-worker executor (see section 5.3.3), concurrent saga runs do not cause parallelism at the pipeline level, the second run's stages will simply wait in the backend queue until the first run's current stage completes.

The server polls the backend every 2 seconds via `GET /jobs/{job_id}` to determine when each stage has finished, updating `run.json` accordingly after each step.

5.3.3 Concurrency in backend requests

The backend uses a `ThreadPoolExecutor` with **a single worker** (`max_workers=1`) to execute pipeline subprocesses. Every pipeline endpoint (`/pipeline/stems`, `/pipeline/prompts`, `/pipeline/music`, `/pipeline/metrics`) immediately returns a `job_id` and submits the actual work to this executor. Jobs are therefore executed **strictly one at a time**, in submission order.

This design is intentional and motivated by two constraints:

- **GPU memory.** The pipeline stages, and especially stage 3 (music generation), load large model weights onto the GPU. Running two stages concurrently risks out-of-memory errors. Serialising execution guarantees that the GPU is fully available to each stage.
- **Pipeline consistency.** Each stage reads outputs written by the previous one. Allowing two stages of the same run to execute concurrently could result in one stage reading incomplete or partially written data from the other. The single-worker executor eliminates this risk entirely.

Job state (`pending`, `running`, `done`, `error`) is tracked in an in-memory dictionary inside the backend process. This means job history is lost if the backend container is restarted. A

migration of the job store to a persistent solution such as Redis [28] is discussed in chapter 9.2 Limitations and Future Work.

5.3.4 Managed saga vs. direct step execution

The two ways to trigger pipeline execution differ significantly in how they handle ordering and state consistency.

5.3.4.1 Managed saga

When a run is started via `POST /pipeline/full`, the server spawns an `asyncio` background task running the saga orchestrator. The saga submits each pipeline stage to the backend one at a time and, after each submission, enters a polling loop:

```
POST /pipeline/stems --> job_id
    loop every 2s: GET /jobs/{job_id} until status == done | error
    update run.json step_status["stems"] = "done"
POST /pipeline/prompts --> job_id
    loop every 2s: GET /jobs/{job_id} until status == done | error
    update run.json step_status["prompts"] = "done"
...
```

Stage $N + 1$ is never submitted until stage N has been confirmed complete by the backend. If any stage returns `error`, the saga stops immediately, records the failed step and the exception message in `run.json`, and sets the overall run status to `error`. This guarantees that:

- Each stage always finds the outputs of the previous stage fully written to disk before it starts.
- `run.json` is kept consistent with the actual state of execution at all times.
- Failures are caught, attributed to the correct step, and surfaced to the caller via `GET /runs/{run_id}`.

A sequence diagram showing the execution of a full managed pipeline is visible in the next page.

5.3.4.2 Direct step proxies

The direct step endpoints (`POST /pipeline/stems`, `POST /pipeline/prompts`, `POST /pipeline/music`, `POST /pipeline/metrics`) are simple fire-and-forget proxies. The server forwards the request to the backend and immediately returns the `job_id` to the caller. There is no polling, no ordering check, and no `run.json` update of any kind.

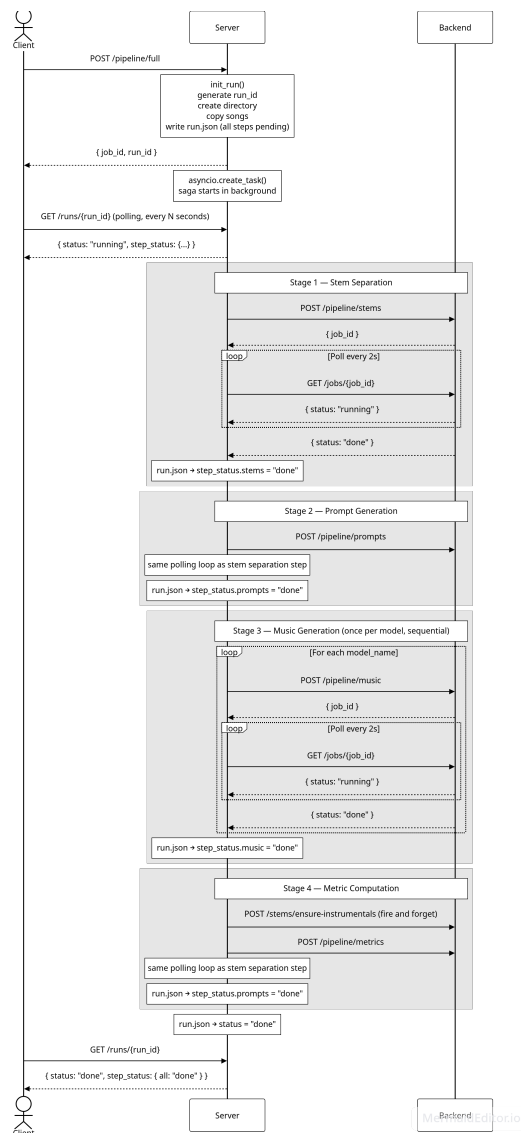


Figura 5.1: Full pipeline execution sequence diagram

The backend's single-worker executor still prevents two stages from executing simultaneously (they are queued in FIFO order), but this is the *only* safety mechanism in place. If a user submits `stems` and `prompts` back-to-back via the proxy endpoints, both jobs are enqueued immediately: `prompts` will start as soon as `stems` finishes on the executor, with no verification that `stems` produced valid output.

This means the direct proxies offer **no consistency or integrity guarantees**: it is entirely the caller's responsibility to poll `GET /jobs/{job_id}` and confirm each step is complete before submitting the next one. They are intended for manual intervention and debugging, not for normal pipeline execution. The UI exclusively uses `POST /pipeline/full`.

Capitolo 6

Frontend

The frontend is the core of the application, since it is used for the creation of new generation runs, the forking from existing ones, and for the display of generation results.

6.1 Structure Overview

6.1.1 Svelte and SvelteKit

The application is built on **SvelteKit** [18], a JavaScript framework for the creation of dynamically rendered websites. SvelteKit is extremely lightweight and fast (as the name suggests), thanks to the Svelte compiler. The rendering of websites using Svelte happens by compiling Svelte code to pure, plain JavaScript, improving speed and robustness. Svelte files contain HTML, CSS and JavaScript language all-in-one, and this significantly reduces the learning curve.

6.1.2 Routing

SvelteKit works on a filesystem-based router, where each route is a directory in the codebase. The structure of the project is the following:

```
client/src/
|-- routes/
|   |-- +layout.svelte, +page.server.js, +page.svelte
|   |-- style.css
|   |-- [run_id]
|   |   |-- in-progress/
|   |   |   |-- +page.server.js, +page.svelte
|   |   |-- results/
|   |   |-- ...
|   |-- +page.server.js, +page.svelte
```

To dynamically render parameters, SvelteKit offers the possibility to name a folder with square brackets `[]`. The use-case of this in the project, as the client filesystem structure shows, is to display a run's status, to fork a determined run, and to see the run's results when it's completed.

The code is then chosen based on the `run_id` explicated in the URL. If a run is called "testrun", the URL would then be `/testrun/results`, therefore displaying the Svelte file mentioned above. Parameters are then retrieved in the `+page.server.js` file, and passed to the `+page.svelte` file using the default data prop.

SvelteKit uses an hybrid approach to rendering, utilizing SSR (Server-Side Rendering) for the first page, and CSR (Client-Side Rendering) for the following page renders. This improves SEO and perceived performance of the initial page load. Client-side rendering then takes over and updates the page without having to re-render common components, which is typically faster and eliminates a flash when navigating between pages.

6.2 UI/UX

The dashboard's design was built with real use cases in mind. The dashboard will be mainly used by experts, who prefer functionality and efficiency over aesthetics. This led to a functional-focused design while keeping it visually appealing, minimizing animations and moving parts.

Inspiration was mainly taken from websites with a similar purpose, having AI-Generated content and/or a ChatBot as interactions. The dashboard is heavily inspired by Anthropic's Claude and Suno AI's music generation websites, both featuring colourful and modern designs with a serif typeface and straight-forward and intuitive interfaces.

The chosen typeface for the dashboard is DM Serif Text:

DM Serif Text

This does a wonderful job at being perfectly legible and stylish while conveying a more human feel to an otherwise synthetic design.



Figura 6.1: Colour Palette

The main colour palette was chosen to be cold purple-based, for its symbolism of wisdom and mystery, perfectly encapsulating the main concept of the dashboard.

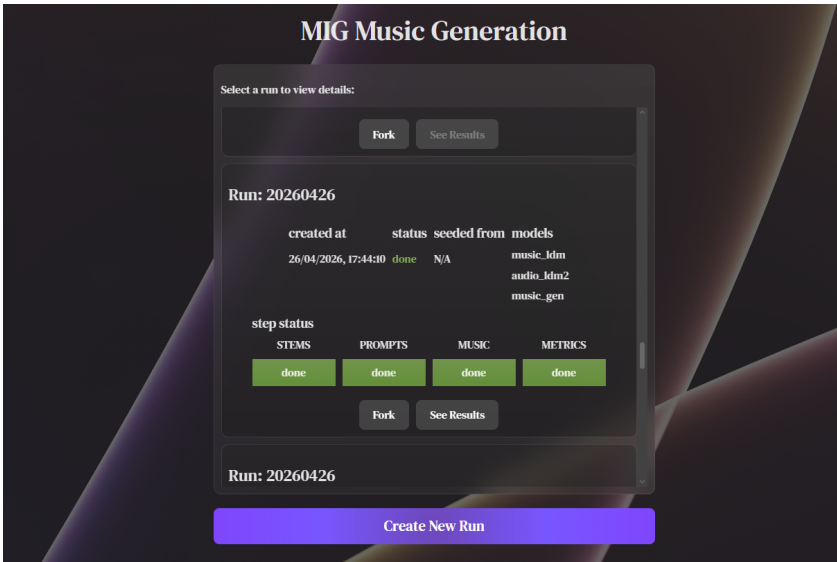


Figura 6.2: Dashboard Home Page

The structure of the webpages was built to be intuitive and easy to navigate, with no distractions and only the necessary information for each page.

6.3 Data Visualization

6.3.1 Design

The results are displayed using bar graphs, and are divided in two main parts: **Model Comparison**, with different models' performance, and **Generation Comparison**, showing the different generations of a certain model.

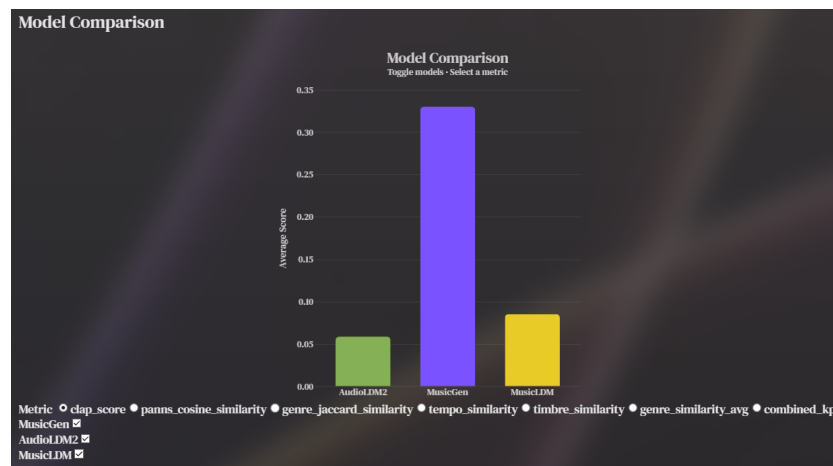


Figura 6.3: Results Graph Mock-up

Every Graph was generated using Vega-Altair [25], which will be explained more in depth on the following subsection. This library allows the implementation of filters, which seamlessly integrates with the display of the results on the dashboard. All calculated final metrics are displayed on the same graph, with buttons for switching efficiently between them.

6.3.2 Graph Generation

As mentioned previously, graphs were generated using the Vega-Altair Python library. Vega-Altair is a declarative statistical visualization library for Python, and offers a powerful and concise grammar that enables you to quickly build a wide range of statistical visualizations. It is partially based on Matplotlib, but offers more customization and interactive features.

Every graph is generated in the server container, and embedded into the website using Vega-Embed, permitting direct interaction from the dashboard. Graphs are pre-generated and sent as a JSON Response to the frontend, with interactive buttons to filter results or show different metrics.

6.4 Backend Communication

6.4.1 API Fetching

Every data shown in the Dashboard is collected by fetching the API exposed by the `mig_server` container, and processed in the `+page.server.js` files. Svelte provides an internal fetching function, which behaves identically to the native `fetch` web API with a few additional features:

- It can make relative requests on the server (ordinarily, `fetch` requires a URL with an origin when used in a server context).

- Internal requests (e.g. for `+server.js` routes) go directly to the handler function when running on the server, without the overhead of an HTTP call.
- During server-side rendering, the response will be captured and inlined into the rendered HTML by hooking into the `text`, `json` and `arrayBuffer` methods of the `Response` object.

This combined with SvelteKit routing, permits the retrieval of run data from the URL of the website thanks to props.

6.5 Dashboard Screenshots

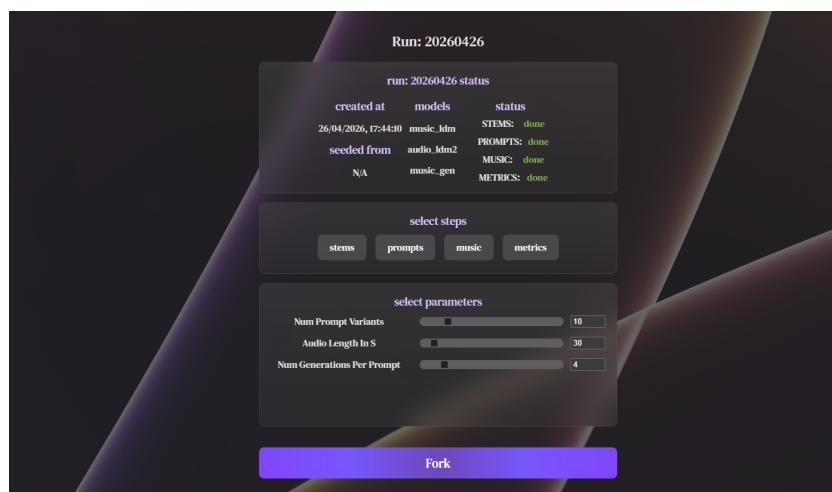


Figura 6.4: Fork Page

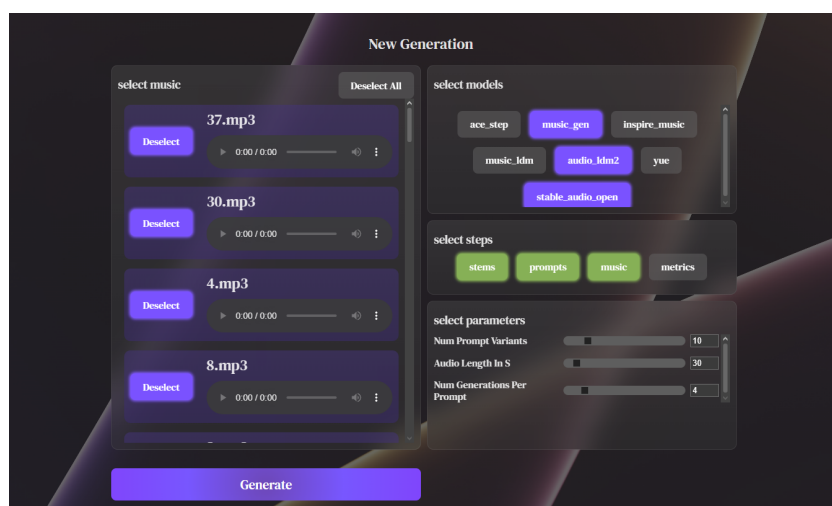


Figura 6.5: Enter Caption

Capitolo 7

Integration and Deployment

The team was composed of two developers. Fabian Tomasovic worked mostly on the frontend, while Nicola Romano worked primarily on the backend and DevOps.

The workload was divided into tasks completed during bi-semesteral sprints. We used Jira to create a backlog, subdivide the work into sprints, and track progress. The tasks ranged from research, development, and testing of the product. The set of tasks to complete inside a sprint was chosen with the project relator, and the status was reported during a weekly meeting.

7.1 Continuous Integration

Given that the developers worked on the same codebase, which could only be partially tested locally, it was necessary from the start of the project to establish a development workflow that enabled fast code integration and deployment to a testing environment, well separated from the production environment.

For this purpose two distinct environments were created on the server. Their differences are discussed in detail in chapter 7 section 2: Continuous Deployment.

We leveraged a bash script (`sync-staging.sh`) that enabled direct code synchronisation to the staging environment on the server, bypassing any interaction with Git or the deployment pipeline, and allowing developers to iterate quickly in a safe, dedicated environment. Tested code could then be committed and pushed to the repository. More information about the repository structure can be found in chapter 4 section 2: Repository Structure.

7.1.1 Version Control System

For source code management and CI/CD pipelines the project uses **GitLab** [27], a platform that combines version control, deployment automation, package management, and team collaboration in a single tool. GitLab served as the central instrument for the DevOps ap-

proach: it improved change traceability, facilitated team collaboration, and enabled secure, consistent integration and deployment processes.

7.1.1.1 Branching strategy

Two long-lived branches were defined:

- `main`: stable branch dedicated to production releases. A push to `main` triggers the production pipeline.
- `dev`: central development branch from which feature branches are derived. A push to `dev` triggers the staging pipeline.

Developers work on short-lived feature branches cut from `dev`. Once a feature is validated in staging it is merged into `dev`, and when a set of features is considered stable, `dev` is merged into `main` to trigger a production release.

7.1.1.2 Protected environment variables

Several environment variables are configured inside the GitLab project and are marked both *protected* and *masked*: they are never printed in pipeline logs and cannot be modified by unauthorised users. They are accessible only to GitLab Runners executing on Linux environments that are enabled for the pipeline. These variables include:

- Path constants for staging and production directories on the host (`STAGING_PATH`, `PROD_DEPLOY_PATH`).
- Base URLs for health-check and smoke-test steps (`STAGING_URL`, `PRODUCTION_URL`).
- Docker Compose project names used to namespace the two stacks on the same host (`COMPOSE_PROJECT_STAGING`, `COMPOSE_PROJECT_PRODUCTION`).

7.2 Continuous Deployment

The CI/CD pipeline is defined in `.gitlab-ci.yml` and consists of four sequential stages: `unit-test`, `build`, `deploy`, `test`: executed automatically by a self-hosted GitLab Runner tagged `spark`, running directly on the NVIDIA Spark server. Two independent pipeline tracks are defined: one for `dev` (staging) and one for `main` (production). Each track runs through the same three stages but targets a different Docker Compose stack and a different directory on the host.

7.2.1 Staging pipeline (dev branch)

7.2.1.1 unit-test

Runs the `pytest` unit-test suite located in `tests/unit/` against a lightweight virtual environment that contains no ML dependencies.

Dependency isolation. Because the production backend imports heavy packages (PyTorch, librosa, torchaudio, etc.) that are impractical to install in a CI runner, `conftest.py` intercepts collection and replaces every such package with either a `types.ModuleType` stub or a `MagicMock` before any test module is loaded. Packages that expose submodule imports (e.g. `torch.nn.functional`) receive real `ModuleType` objects rather than mocks, because Python's import machinery requires a genuine package hierarchy for dotted-name imports.

Covered modules. Two test modules are executed.

`test_run_management.py` validates the run-metadata persistence layer in isolation: the `save_run_metadata`, `load_run_metadata`, and `get_run_path` helpers are re-implemented verbatim inside the test file so that no transitive import of `endpoints.py` is needed. The tests cover:

- round-trip serialisation, data written by `save_run_metadata` is byte-for-byte equal after `load_run_metadata`;
- missing-run resilience, loading a non-existent `run_id` returns an empty dict rather than raising;
- path invariants, the resolved path always starts with `runs_dir` and contains the `run_id` segment;
- schema constraints, `step_status` must contain exactly the four keys `stems`, `prompts`, `music`, and `metrics`, each holding one of the five valid status strings (`pending`, `running`, `done`, `error`, `seeded`);
- `run_id` format

`test_endpoints_models.py` targets the Pydantic request models defined in `backend/src/api.py` (`StemsRequest`, `MusicRequest`,...). Because FastAPI ships an internal module also named `api` that Python resolves before `backend/src/api.py` regardless of `PYTHONPATH` ordering, the module is loaded by file path via `importlib.util.spec_from_file_location`. If loading fails (ML transitive deps absent), all tests in the file are skipped automatically; coverage for the same models is then provided by the schema content checks in `test:staging`. When the module is loadable the tests assert field defaults and that missing required fields raise a `Pydantic.ValidationError`.

Shared fixtures. The `tmp_run_dir` fixture in `conftest.py` materialises a minimal but structurally complete run directory, including the four step sub-directories and a pre-populated `run.json`, inside pytest's temporary path, so every test that needs a run on disk receives an isolated, reproducible tree with no side-effects.

7.2.1.2 build:staging

Builds all Docker images using the base `docker-compose.yml` overlaid with `docker-compose.staging.yml`. The `-pull` flag ensures that base images are refreshed from the registry before each build.

7.2.1.3 deploy:staging

Synchronises the repository checkout to `STAGING_PATH` on the host using `rsync`, excluding the `.git` directory, build artefacts, and virtual environments.

It then brings the staging stack up with `docker compose up -d --no-build --remove-orphans`: `--no-build` ensures only the images built in the previous job are used, and `--remove-orphans` cleans up containers from removed services.

Finally, `wait_for_health.sh` blocks the job until the server's `GET /health` endpoint responds, guaranteeing the stack is ready before the test job begins.

7.2.1.4 test:staging

Runs `test_api.sh` against `STAGING_URL`. This is a smoke-test suite that verifies the most critical endpoints are reachable and return the expected HTTP status codes and body content.

The tests currently cover:

- `GET /health`: checks both `status: ok` and `service: server`.
- All four `GET /api/pipeline/{step}/schema` endpoints, verifies the JSON Schema is returned ("`properties`" key present).
- `GET /api/jobs/nonexistent-job-id`: verifies that the API returns 404 for unknown job IDs.

If any assertion fails the script exits with a non-zero code, failing the GitLab job and blocking any downstream merge to `main`.

7.2.2 Production pipeline (main branch)

The production track mirrors the staging track with two deliberate differences:

- **No-cache build.** `build:production` passes `-no-cache` in addition to `-pull`, forcing a fully clean image build. This eliminates any risk of stale layer cache reaching production, at the cost of a longer build time.
- **Separate deploy path.** Files are synchronised to `PROD_DEPLOY_PATH` (`prod-deployment/`), a lean non-Git directory that contains only the production `.env`, `input/`, and `output/` directories, as described in chapter 4.3 Host Server Architecture.

Both stacks run simultaneously on the same host, isolated by their Docker Compose project names (`mmgd_staging` and `mmgd_prod`) and exposed on different ports (8080 for staging, 80 for production).

7.2.3 On-Premises Server Requirements

Other than the architecture described at chapter 4 section 3: Host server architecture, there are some other requirements to prepare the host server to be productive.

First a csv file named `models.csv` should live under `output/runs` and should be populated following this structure:

```
model_name,display_name,description
audio_ldm2,AudioLDM2,Latent diffusion model for audio generation
music_gen,MusicGen,Meta's autoregressive music generation model
...,...,...
```

Secondly a GitLab runner should be installed with docker super user privileges, and should be correctly connected to the GitLab instance where the repository lives.

7.3 Development workflow

The day-to-day workflow is designed to keep the production environment completely isolated from development activity while still allowing fast iteration on the server, where the GPU and model weights are available.

1. **Local development.** Developers work on a feature branch cut from `dev`. Code that does not require GPU or model inference can be partially tested locally.
2. **Fast server testing via `sync-staging.sh`.** When GPU-dependent behaviour must be verified, the developer runs `./sync-staging.sh`. This script uses `rsync` to push the local working tree directly to `staging-env/` on the server (SSH key required), intentionally excluding `.git`, `input/`, `output/`, `.env`, and the model directory to avoid overwriting persistent data. The developer then manually restarts the affected containers on the server to pick up the changes. This loop, edit locally, sync, restart, verify,

is the primary inner development cycle and deliberately bypasses Git to maximise iteration speed.

3. **Commit and push to `dev`.** Once the feature behaves correctly in staging the developer commits and opens a merge request into `dev`. The push triggers the staging CI/CD pipeline automatically: the stack is rebuilt, redeployed, and the smoke-test suite runs. A failed pipeline blocks the merge.
4. **Merge to `main` for production release.** When a set of features on `dev` is considered stable, `dev` is merged into `main`. This triggers the production pipeline, which performs a clean no-cache build and deploys to `prod-deployment/`.

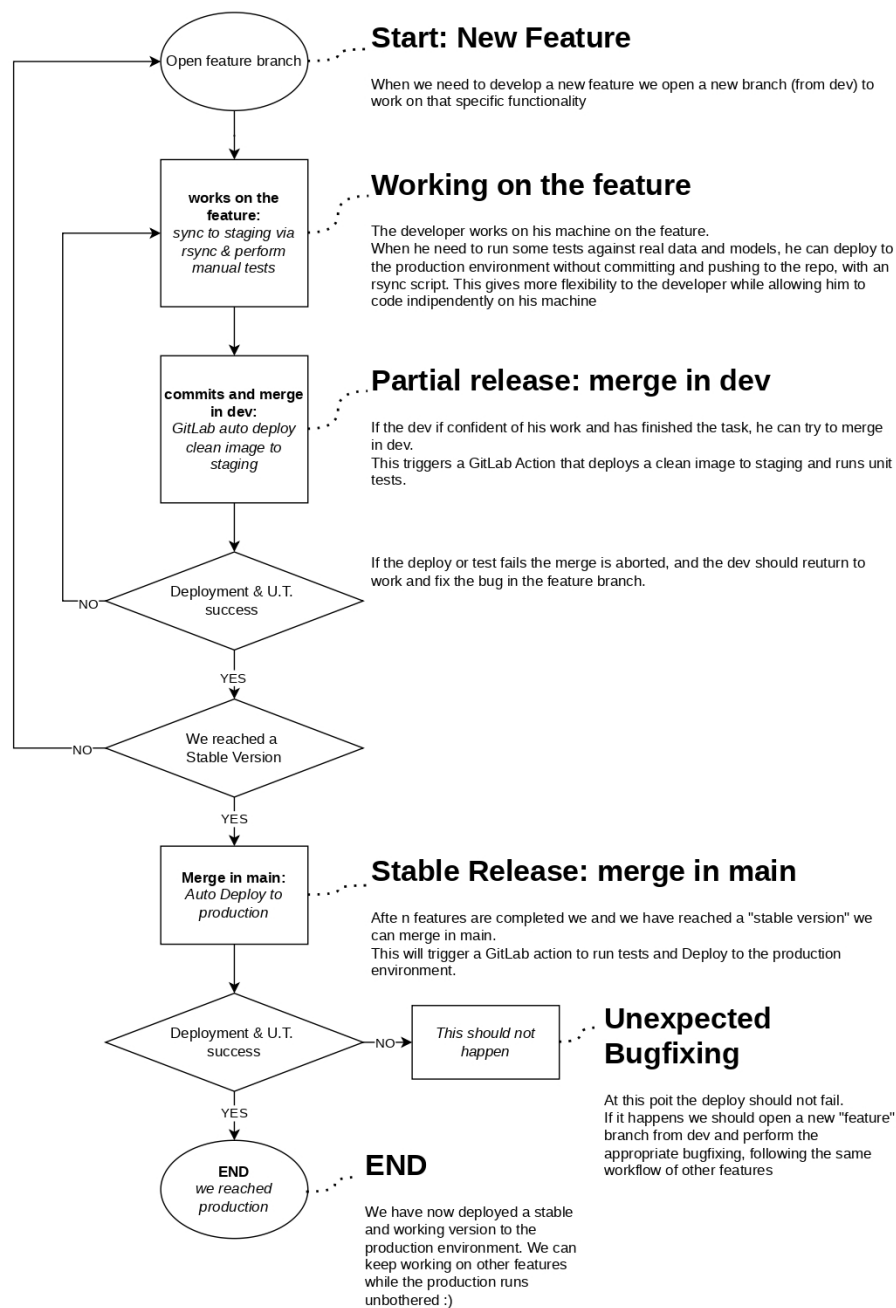


Figura 7.1: Development workflow flowchart

Capitolo 8

Pipeline Update

The system was originally developed on an x86 workstation by the project relator. During the integration phase, when the pipelines were deployed on the NVIDIA Spark server, an **aarch64 (ARM64)** machine, a critical incompatibility surfaced in the metrics pipeline, requiring a migration of the genre classification component. This chapter documents that migration and the additional edge cases that were identified and resolved during development.

8.1 Essentia to PANNs migration

8.1.1 Root cause

The original metrics pipeline relied on **Essentia-TensorFlow** [29] for genre classification. Essentia is a C++ library with Python bindings; its TensorFlow integration ships as a pre-built wheel on PyPI for x86_64, but **no pre-built wheel exists for aarch64**.

We were able to build Essentia from source on the ARM server and to install TensorFlow independently, but the two could not be linked together: the Essentia build system requires TensorFlow to be present at compile time to enable its TensorFlow operator support, and this step failed consistently on the ARM toolchain. Compiling a custom TensorFlow from source to satisfy the Essentia build would have required a prohibitive amount of time and introduced an unmaintainable dependency chain.

The decision was therefore taken to replace Essentia entirely with **PANNs** (*Pre-trained Audio Neural Networks*), a pure-Python inference library available on PyPI for all platforms, including aarch64.

8.1.2 What Essentia provided

Essentia powered three independent genre classifiers, each with a different embedding backbone and label taxonomy:

- **discogs519**: 519 classes in Genre--Subgenre hierarchical format, computed from MAEST embeddings.
- **discogs400**: 400 classes in Genre--Subgenre format, computed from EffNet embeddings.
- **mtg_jamendo**: 195 flat genre classes, computed from EffNet embeddings.

The three models provided an ensemble signal: genre scores were averaged across classifiers to produce a more robust genre representation, and the hierarchical Genre--Subgenre labels enabled separate treatment of broad genres and fine-grained subgenres.

8.1.3 What PANNs provides

PANNs exposes a **single model** trained on the AudioSet ontology with **527 flat classes**. The model outputs a probability for each class given an audio clip. There is no ensemble, no subgenre hierarchy, and the label space covers not only music genres but also speech, noise, and environmental sounds.

8.1.4 Migration steps and intermediate shim

The first migration pass retained the 8-value return signature of the original `predict_genre()` function and populated the existing `discogs519`, `discogs400`, and `mtg_jamendo` columns with identical copies of the PANNs output. While this kept the rest of the pipeline from breaking, it introduced several correctness problems:

- The three `*_avg_genre_scores` columns carried identical values, providing no ensemble signal whatsoever.
- `genre_score_similarity` averaged three identical cosine values, making the operation a no-op.
- Jaccard similarity was computed via `sklearn.preprocessing.MultiLabelBinarizer` [30] fitted on a flat string, which treated each *character* as a label, a misuse of the sklearn API producing meaningless results.
- `genre_subgenre_predictions_file` was accepted as a parameter but never written to disk, since PANNs has no subgenre concept.

All of the above was removed in the final migration. The codebase was cleaned up to be PANNs-native.

8.1.5 Current schema

`predict_genre()` now returns exactly two values:

- `panns_top_tags (str)`: the top-5 music-relevant AudioSet labels, comma-separated, used for Jaccard similarity. A music-keyword filter is applied before selecting labels to avoid generic tags such as "Music" or "Singing" dominating the result.
- `panns_scores (list)`: the full 527-dimensional probability vector as [(label, score), ...], unfiltered, used for cosine similarity.

The descriptor CSVs (`descriptors.csv`, `descriptors_on_original.csv`, `descriptors_on_generated_{model}.csv`) now carry `panns_top_tags` and `panns_scores` in place of the six Essentia genre columns.

The similarity columns in `descriptors_evaluation_of_generated_{model}.csv` are:

- `genre_jaccard_similarity_{model}`: Jaccard similarity on the `panns_top_tags` tag sets.
- `panns_cosine_similarity_{model}`: cosine similarity on the full 527-dimensional score vectors. Both vectors must cover all 527 classes in the same label order, a contract enforced by a shape check inside `genre_score_similarity()`.

The `combined_kpi` formula was updated accordingly to replace the Essentia genre columns with the PANNs equivalents (see Chapter 2, Section 2.2 for the full formula).

8.1.6 What PANNs cannot replicate

The migration involved unavoidable trade-offs that are worth documenting:

- **No ensemble.** Essentia's three independent classifiers produced an averaged genre signal. PANNs uses a single forward pass; there is no cross-model averaging.
- **No subgenre hierarchy.** The Genre--Subgenre label format is lost. All genre information is now flat AudioSet tags. The functions `extract_essentia_genres_subgenres()`, `build_genre_subgenre_df()`, `compute_avg_score_per_genre()`, and `process_model_genres()` are retained as dead code for reference, marked `ESSENTIA_LEGACY`.
- **Broader label space.** AudioSet covers 527 classes including non-music sounds. The top-5 labels for a music file may include environmental or vocal labels, which is why the music-keyword filter is applied for `panns_top_tags`.

8.2 Handling edge cases

8.2.1 Missing instrumental stems

The metrics pipeline requires an `_instrumental.wav` file for each input song, the original track without vocals, to use as the reference for audio feature extraction. This file is produced by the stem separation stage (stage 1), which sums the non-vocal stems (bass, drums, other) into a single instrumental track.

In practice, the instrumental file could be absent if a run was seeded from a prior run that predated the `ensure-instrumentals` logic, or if the stem separation stage had been interrupted before the merge step. To handle this, the server calls

`POST /stems/ensure-instrumentals` before submitting the metrics stage. This endpoint scans the `stems/` directory of the run and, for each song that has a `_bass.wav` but no `_instrumental.wav`, reconstructs the instrumental by summing the available stems. If the peak amplitude of the mix exceeds 1.0, the result is normalised to prevent clipping. Songs that already have an instrumental file are skipped without recomputing.

8.2.2 Generated audio file layout

The music generation pipeline can write output files in two different directory layouts depending on the generation model and whether multiple generations per prompt were requested:

- **Flat layout:** all generated files in the model output directory directly:
`music/open_generation/{model}/{file_id}.wav`
- **Subfolder layout:** one subdirectory per generation index:
`music/open_generation/{model}/generation_{n}/{file_id}.wav`

The metrics pipeline's `attribute_extractor()` detects the layout at runtime by checking whether the `generation_{n}` subdirectory exists before constructing the file path, falling back to the flat layout if not found. This avoids hardcoding assumptions about the generation model's output structure.

8.2.3 Wikipedia API network failures

Several methods in `music_prompt_attributes.py` query the Wikipedia API to enrich song metadata. The `wikipedia` Python library does not handle transport-level failures internally: when the MediaWiki API returns an empty body, due to a transient network error or rate-limiting, the library raises a `requests.exceptions.JSONDecodeError`, which falls outside the `PageError` and `DisambiguationError` types that the existing `except` blocks covered. The fix was applied at two levels: a monkey-patch on the library's internal `_wiki_request` function converts any such exception into a `WikipediaException` at the source, and the

except clauses at each call site were widened to `Exception` so that a failed lookup is logged as a warning and treated as a missing page rather than a fatal error.

8.2.4 Ollama response failures in prompting services

The methods in `prompting_services.py` that call the local Ollama LLM share a common failure mode: the model occasionally returns an empty string instead of the expected output. Depending on the method, this produced one of two error categories. Methods that parsed the response as JSON (e.g. `get_purpose_mood_theme_from_ollama`) raised a `JSONDecodeError` which, after exhausting a fixed number of retries, was re-raised as a `ValueError` and propagated uncaught up the call stack. Methods that applied string operations directly to the response (e.g. `get_epoch_info_from_ollama`, which called `.splitlines()[0]`) raised an `IndexError` immediately. In both cases the entire prompts step was terminated. The fix followed the same pattern throughout: an explicit empty-response guard was added before any parsing or string operation, and hard re-raises were replaced with a warning log and a `None` return, so a single failing LLM call causes only that field to be absent rather than aborting the pipeline.

Capitolo 9

Conclusions

9.1 Summary

This work set out to transform a research pipeline that existed only as a collection of manually executed scripts into a deployable, reproducible, and usable benchmarking platform. Starting from a codebase designed for a single workstation, we engineered a multi-container Docker application, designed and implemented a two-layer HTTP API with structured run management, built an interactive web dashboard, and established a CI/CD pipeline that enforces a clean separation between a staging and a production environment.

Along the way, a critical platform incompatibility was resolved: the genre classification component of the metrics pipeline was migrated from Essentia-TensorFlow, which could not be compiled on the aarch64 deployment server, to PANNs, a pure-Python alternative that runs on all supported platforms. Several edge cases in the prompt generation and metrics stages were also identified and fixed, making the pipeline significantly more robust than it was at the start of the project.

The result is a platform that allows researchers to submit audio tracks, configure which generation models to benchmark, and receive a structured set of evaluation metrics, CLAP score, tempo similarity, timbre similarity, and PANNs cosine similarity, presented through an interactive dashboard. The platform supports experiment reproducibility through run seeding, safe concurrent execution through GPU serialisation, and reliable deployment through automated CI/CD.

9.1.1 Results

To validate the end-to-end pipeline, a complete benchmark run was executed using a corpus of six reference tracks across three generation models: **MusicGen** [2], **AudioLDM2** [4], and **MusicLDM** [6]. Each model produced two generations per track, for a total of 36 generated audio files evaluated against the reference corpus.

It is important to frame these results correctly: this run constitutes a functional validation of the platform rather than a statistically conclusive evaluation of the models. Six reference tracks is a small corpus, and the scores should be interpreted as indicative rather than definitive. The primary purpose of this run was to verify that all four pipeline stages execute correctly end-to-end, that metrics are computed and aggregated as expected, and that the results surface meaningfully in the dashboard.

With that caveat in mind, the results show a clear differentiation between the three models. **MusicGen** achieved the highest scores across most metrics, with a CLAP score of 0.330, a timbre similarity of 0.833, and a combined KPI of 0.663. **AudioLDM2** and **MusicLDM** scored comparably on most dimensions but significantly lower on CLAP score (0.059 and 0.085 respectively), suggesting that their outputs are less semantically aligned with the text prompts used in this run. PANNs cosine similarity was high for all three models (0.83–0.90), indicating that the generated audio broadly preserves the genre characteristics of the reference tracks regardless of the model used.

The generation comparison shows that scores are nearly identical across the two generations for AudioLDM2 and MusicLDM, while MusicGen shows a marginal variation in combined KPI between generation 1 (0.661) and generation 2 (0.662), which is within noise for a corpus of this size.

These results demonstrate that the platform produces coherent, interpretable output. Researchers will be able to leverage this infrastructure to conduct more comprehensive evaluations, with larger corpora, more generation variants, and the full set of seven supported models, once the remaining dependency environments are stabilised as described in section 9.2. The structured run format, the seeding mechanism, and the interactive dashboard are specifically designed to make that kind of systematic, iterative experimentation efficient.

9.2 Limitations and future work

9.2.1 Dependency management in `mig_backend`

The `mig_backend` container currently relies on **pip** to manage the dependencies of its four pipeline virtual environments. While the `requirements.txt` files pin direct dependencies to guarantee reproducibility, pip provides no built-in mechanism to resolve and lock the full transitive dependency graph. This means that indirect dependencies, packages pulled in by packages, are not pinned, and may drift silently between builds as upstream releases are published. Over time, this creates significant technical debt: updating a single direct dependency can produce cascading, difficult-to-diagnose conflicts at levels that pip does not surface clearly. The `mig_server` container, by contrast, already uses **uv** [17], which generates a `uv.lock` file that pins the entire dependency graph deterministically. Migrating

the backend virtual environments to uv would eliminate this risk, reduce build times through uv's parallel resolver, and make dependency upgrades significantly more manageable.

9.2.2 Incomplete model support

A direct consequence of the pip limitation is that, while the codebase already integrates wrappers for seven generation models, MusicGen, MusicLDM, AudioLDM2, Stable Audio Open, InspireMusic, YUE, and ACE-Step, only the first three are currently backed by up-to-date, validated dependency sets. The remaining four models are present in the codebase and in the prompt generation stage, but their virtual environments have not been brought to a working state on the aarch64 deployment platform. A migration to uv, with its cleaner resolution model, would provide the right foundation to validate and stabilise the remaining environments and unlock the full set of supported models for benchmarking.

9.2.3 Monolithic backend container

The `mig_backend` container hosts all four pipeline stages, each in its own virtual environment. While the virtual environment separation prevents dependency conflicts at the Python level, the result is a single, very large container image that must be fully rebuilt whenever any pipeline changes. Each virtual environment contains a full copy of PyTorch and other heavy ML libraries, contributing to an image size that is disproportionate to what any single pipeline stage requires. The natural evolution is to give each pipeline stage its own dedicated container, `mig_stems`, `mig_prompts`, `mig_music`, `mig_metrics`, each carrying only the dependencies it actually needs. This would reduce individual image sizes, enable independent rebuilds and deployments of pipeline stages, and remove the need for virtual environments within the backend entirely, since each container would already provide an isolated environment.

9.2.4 Inversion of the job execution flow with Redis and Celery

The current architecture follows a push model: the `mig_server` submits jobs directly to the `mig_backend` via HTTP, then polls the backend every two seconds until the job completes. This creates a tight coupling between the server and the backend, requires the server to maintain polling loops as long-lived `asyncio` tasks, and means that job state, currently held in an in-memory dictionary inside the backend process, is lost if the backend container is restarted.

Once pipeline stages are separated into individual containers, a cleaner architecture becomes possible: the server posts job requests to a **Redis** [28] queue, and each pipeline container runs a **Celery** [31] worker that consumes tasks from its own dedicated queue. This inverts the call flow from push to pull, decoupling the server from the pipeline containers entirely. GPU serialisation, currently enforced by the single-worker `ThreadPoolExecutor`:

would be delegated to Celery's concurrency configuration, with each pipeline worker limited to one concurrent task. Job state would be persisted in Redis, surviving container restarts and providing a durable audit trail of past executions. The server would transition from polling to subscribing to job status updates, eliminating the two-second polling interval.

9.2.5 API authentication and access control

The current server API has no authentication layer. All endpoints, including those that delete runs or trigger pipeline executions, are accessible to any client that can reach the server port. In a research environment shared among multiple users, this creates a risk of accidental interference between concurrent experiments. Adding an authentication mechanism, even a simple API-key-based one, would allow access to be scoped per user and prevent unauthorised pipeline executions or deletions.

9.2.6 Real-time log streaming

Run logs are currently accessible through a polling GET `/runs/{run_id}/log` endpoint, which requires the client to repeatedly request the log file to observe progress. A natural improvement would be to expose log output as a **WebSocket** or a **Server-Sent Events** stream, allowing the dashboard to display pipeline output in real time without repeated HTTP requests. This would also remove the need for the client-side polling loop currently implemented in the in-progress page.

9.2.7 Frontend Error Prevention

As explained previously, the Frontend works by routing based on the current URL. This is missing handling non-existing Run IDs, which would benefit by having a redirection to a missing page.

Bibliografia

- [1] Docker Inc. Docker: Accelerated container application development, 2024. Accessed: 2025.
- [2] Jade Copet, Felix Kreuk, Itai Gat, Tal Remez, David Kant, Gabriel Synnaeve, Yossi Adi, and Alexandre Défossez. Simple and controllable music generation. *Advances in Neural Information Processing Systems*, 36, 2023.
- [3] Alexandre Défossez, Jade Copet, Gabriel Synnaeve, and Yossi Adi. High fidelity neural audio compression. *arXiv preprint arXiv:2210.13438*, 2022.
- [4] Haohe Liu, Yi Yuan, Xubo Liu, Xinhao Mei, Qiuqiang Kong, Qiao Tian, Yuping Wang, Wenwu Wang, Yuxuan Wang, and Mark D. Plumbley. AudioLDM 2: Learning holistic audio generation with self-supervised pretraining. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2024.
- [5] Haohe Liu, Zehua Chen, Yi Yuan, Xinhao Mei, Xubo Liu, Danilo Mandic, Wenwu Wang, and Mark D. Plumbley. AudioLDM: Text-to-audio generation with latent diffusion models. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2023.
- [6] Ke Chen, Yatian Wang, Xingjian Du, Bilal Siddique, Jiachen Luo, Yizhi Li, Gus Xia, Minz Won, Shengchen Li, Taylor Berg-Kirkpatrick, and Shlomo Dubnov. MusicLDM: Enhancing novelty in text-to-music generation using beat-synchronous mixup strategies. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2024.
- [7] Zach Evans, CJ Carr, Josiah Taylor, Scott H. Hawley, and Jordi Pons. Fast timing-conditioned latent audio diffusion. *arXiv preprint arXiv:2402.04825*, 2024.
- [8] InspireMusic Team. InspireMusic: Integrating super resolution and score matching for high-fidelity music generation, 2024. Accessed: 2025.
- [9] Ruibin Yuan, Hanfeng Lin, Yi Wang, Zeyue Tian, Shangda Wu, Tianhao Shen, Ge Zhang, Yizhi Li, Haoran Luo, Siwei Wu, Guangze Li, Aosen Wang, Lijun Yu, Emmanouil Benetos, Yike Guo, Chenghua Lin, Wenhua Chen, Gus Xia, Jie Fu, Roger

- Dannenberg, Wei Xue, Shiyin Lu, Jian Yang, Chao Zhang, Yike Guo, and Feilong Chen. YuE: Open music foundation models for full-song generation. *arXiv preprint arXiv:2503.08638*, 2025.
- [10] ACE-Step Team. ACE-Step: A step towards music generation foundation model, 2025. Accessed: 2025.
- [11] Yusong Wu, Ke Chen, Tianyu Zhang, Yuchen Hui, Taylor Berg-Kirkpatrick, and Shlomo Dubnov. Large-scale contrastive language-audio pretraining with feature fusion and keyword-to-caption augmentation. In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2023.
- [12] LAION AI. LAION-CLAP: Contrastive language-audio pretraining, 2023. Accessed: 2025.
- [13] Brian McFee, Colin Raffel, Dawen Liang, Daniel P.W. Ellis, Matt McVicar, Eric Batteberg, and Oriol Nieto. librosa: Audio and music signal analysis in python. In *Proceedings of the 14th Python in Science Conference (SciPy)*, 2015.
- [14] George Tzanetakis and Perry Cook. Musical genre classification of audio signals. *IEEE Transactions on Speech and Audio Processing*, 10(5):293–302, 2002.
- [15] Qiuqiang Kong, Yin Cao, Turab Iqbal, Yuxuan Wang, Wenwu Wang, and Mark D. Plumbley. PANNs: Large-scale pretrained audio neural networks for audio pattern recognition. In *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, volume 28, pages 2880–2894, 2020.
- [16] Sebastián Ramírez. FastAPI, 2024. Accessed: 2025.
- [17] Astral. uv: An extremely fast python package installer and resolver, 2024. Accessed: 2025.
- [18] Rich Harris and Svelte Contributors. Svelte / SvelteKit, 2024. Accessed: 2025.
- [19] Alexandre Défossez, Nicolas Usunier, Léon Bottou, and Francis Bach. Hybrid spectrogram and waveform source separation. *arXiv preprint arXiv:2111.03600*, 2021.
- [20] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. *arXiv preprint arXiv:2212.04356*, 2022.
- [21] OpenAI. GPT-4 technical report, 2024. Accessed: 2025.
- [22] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Léo Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le

Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mistral 7B. *arXiv preprint arXiv:2310.06825*, 2023.

- [23] Ollama. Ollama: Run large language models locally, 2024. Accessed: 2025.
- [24] The pandas development team. pandas: Powerful python data analysis toolkit, 2024. Accessed: 2025.
- [25] Jacob VanderPlas, Brian Granger, Jeffrey Heer, Dominik Moritz, Kanit Wongsuphasawat, Arvind Satyanarayan, Eitan Lees, Iliia Timofeev, Ben Welsh, and Scott Sievert. Altair: Interactive statistical visualizations for Python, 2018. Accessed: 2025.
- [26] Docker Inc. Docker Compose, 2024. Accessed: 2025.
- [27] GitLab Inc. GitLab: The devsecops platform, 2024. Accessed: 2025.
- [28] Redis Ltd. Redis: The real-time data platform, 2024. Accessed: 2025.
- [29] Dmitry Bogdanov, Nicolas Wack, Emilia Gómez, Sankalp Gulati, Perfecto Herrera, Oscar Mayor, Gerard Roma, Justin Salamon, José R. Zapata, and Xavier Serra. Essentia: An audio analysis library for music information retrieval. In *Proceedings of the 14th International Society for Music Information Retrieval Conference (ISMIR)*, pages 493–498, 2013.
- [30] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [31] Ask Solem and Celery Contributors. Celery: Distributed task queue, 2024. Accessed: 2025.